Hospital-Bed-and-Operating-Room-Scheduling

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
B.Bharat Sai Reddy - 2520090128
A.Sri Chaitanya - 2520090094

Team Number: 14

Section: 11

Under the guidance of

Mrs. CH. Anitha Chekuri
Assistant Professor



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled “Hospital-Bed-and-Operating-Room-Scheduling” is a record of Bonafide work of B.Bharat Sai Reddy - 2520090128, A.Sri Chaitanya - 2520090094 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

B. Bharat Sai Reddy - 2520090128

A. Sri Chaitanya - 2520090094

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled “Hospital-Bed-and-Operating-Room-Scheduling” is a bonafide work done and submitted by B.Bharat Sai Reddy - 2520090128, A.Sri Chaitanya - 2520090094 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Mrs. CH. Anitha Chekuri, Faculty, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT),** Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

B. Bharath Sai Reddy-2520090128

A. Sri Chaitanya - 2520090094

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

Efficient scheduling of hospital beds and operating rooms is a critical challenge in modern healthcare, where limited resources must be allocated while maintaining high-quality patient care. This project proposes an intelligent scheduling system that assigns patients and surgical procedures to available beds, operating rooms, and time slots based on multiple constraints, including treatment duration, patient priority, staff availability, resource capacity, and scheduling deadlines. Since hospital scheduling is a complex optimization problem that is computationally difficult to solve optimally for large datasets, the proposed system combines network flow techniques for resource allocation with approximation strategies to generate high-quality schedules within practical execution time.

The scheduling engine aims to maximize resource utilization, reduce patient waiting time, minimize operating room idle time, and decrease the overall completion time (make span) while satisfying all operational constraints. The system also generates detailed reports containing bed occupancy, operating room utilization, scheduling efficiency, and constraint violations to support hospital administrators in decision-making. This project demonstrates the practical application of advanced data structures and algorithms, particularly network flow, assignment algorithms, and approximation techniques, to solve real-world healthcare resource optimization problems.

Team Name: Hospital-Bed-and-Operating-Room-Scheduling

Team Members:

1. B.Bharat Sai Reddy - 2520090128
2. A.Sri Chaitanya - 2520090094

2. INTRODUCTION

Hospitals must decide, every day, which facility treats a patient, in which operating room (OR) and at what time the surgery starts, and which bed the patient occupies afterwards. These decisions are tightly coupled: a surgery cannot start without a free, suitably equipped OR and a qualified surgeon, and it should not start unless a bed of the right type (General or ICU) is available for recovery. Doing this by hand becomes error-prone when emergencies arrive, resources are limited and several hospitals share the load.

What the project is. This project, the **Hospital Bed and Operating Room Scheduling System**, is a Java simulation that assigns patients to hospitals, ORs, surgery start times and beds. It exists in two forms that share the same scheduling code: a command-line program that runs on a fixed sample dataset (3 hospitals, 22 beds, 7 ORs, 4 surgeons and 12 patients) and a Spring REST application that stores hospitals, schedules, bed reservations and discharges in a local file-backed H2 database.

Why it matters. The problem combines several classic hard problems (capacity-constrained routing, multiprocessor load balancing, sequencing under time windows and assignment). It therefore gives a realistic setting in which to select, justify and combine the advanced algorithms studied in Data Structures and Algorithms-3 instead of applying a single textbook routine.

How it works. The scheduler is a four-stage pipeline: (1) min-cost maximum flow routes each patient to a capable hospital while respecting separate ICU and General bed capacities; (2) randomized quicksort orders cases by duration and the Longest Processing Time (LPT) heuristic balances them over compatible ORs; (3) subset dynamic programming sequences the cases of each OR around existing bookings, surgeon shifts, priority and waiting time; and (4) the Hungarian algorithm reserves beds for each patient's full stay. A Knuth-Morris-Pratt (KMP) search module finds patients and staff by name, and KPI and schedule reports summarise the outcome.

Scope. This is a classroom simulation using hard-coded data. It is not a live or clinically validated hospital system, and it does not model anesthetists, nurses, equipment or real-time monitoring.

Organisation of the report. Section 3 surveys related work; Section 4 states the problem and objectives; Section 5 lists hardware and software requirements; Section 6 presents the main modules; Section 7 reports the results; Section 8 gives the conclusion and future scope; and Section 9 lists the references.

3. LITERATURE SURVEY

The following previous studies on operating room, patient and bed scheduling, together with the classical algorithms used in this project, were reviewed. Their links are given in the References section.

1. Operating room planning and scheduling: a literature review (Cardoen, Demeulemeester and Beliën, 2010). This widely cited survey in the European Journal of Operational Research classifies OR planning research by problem setting (patient classes, performance measures) and by technical features (solution technique, handling of uncertainty). It shows that elective-patient scheduling dominates the literature and that emergency handling and integrated resource planning remain open areas. It motivated our separation of the problem into routing, room, time and bed stages and our explicit emergency-replanning module [1].

2. Scheduling operating rooms: achievements, challenges and pitfalls (Samudra et al., 2016). This review updates the earlier survey with more recent OR planning and scheduling work and discusses practical difficulties in applying operations-research models in hospitals. It stresses that OR decisions depend on coordinated resources, including rooms, surgeons and downstream beds, which is why our pipeline checks surgeon qualification and bed feasibility before an operation is committed [2].

3. Operating Room Planning and Scheduling (Demeulemeester, Beliën, Cardoen and Samudra, 2013). This book chapter in the Handbook of Healthcare Operations Management describes how OR scheduling involves the coordination of patients, medical staff and hospital facilities, and how the downstream ICU and ward must be available after surgery. It reviews trends in patient type, performance measures and methodology. This supports our decision to couple OR time slots with ICU/General bed capacity [3].

4. A hybrid tabu search algorithm for automatically assigning patients to beds (Demeester et al., 2010). The authors describe a patient admission scheduling algorithm that assigns patients to beds in suitable departments, considering medical needs and preferences while keeping departments balanced. They use a hybridised tabu search and compare it with an integer-programming approach. Their work is a metaheuristic alternative to our exact Hungarian bed assignment [4].

5. Harmony Search Algorithm for the Patient Admission Scheduling Problem (Abu Doush et al., 2018). This paper solves the patient admission scheduling (PAS) problem, assigning patients to beds over a time period while preserving medical requirements and preferences, with a harmony-search metaheuristic. It is compared against nine other methods, including tabu search and simulated annealing, and gives competitive results. It illustrates the heuristic family that we deliberately replaced with a deterministic assignment solver [5].

6. A greedy-tabu approach to the patient bed assignment problem at Hospital Universitario San Ignacio. This study models patient-bed assignment using requirements such as diagnosis and equipment, and uses a greedy algorithm followed by tabu search because exact solution times were long. The reported gap to the optimum is about 6.2% on average over 15 test instances. It highlights the trade-off between solution quality and run time that we address with the O(n^3) Hungarian algorithm on small instances [6].

7. Structural insights and an IP-based solution method for patient-to-room assignment (arXiv, 2024). This paper reviews the patient-to-room assignment literature, noting that Demeester et al. (2010) gave the first formal definition of the problem and that restricting room choices makes the task NP-complete. It then presents an integer-programming method. It confirms that bed assignment is a hard combinatorial problem and that our hand-designed cost matrix is a simplification [7].

8. Classical algorithms used in this project. Kuhn's Hungarian method solves the assignment problem exactly [8]; Graham's analysis of list scheduling underpins the LPT heuristic for multiprocessor load balancing [9]; Knuth, Morris and Pratt gave linear-time string matching [10]; and Edmonds and Karp's shortest-augmenting-path analysis forms the basis of network-flow routing [11]. Our project combines these standard techniques in one hospital workflow.

Research gap. Most of the surveyed work optimises one stage (OR planning, or bed assignment) with metaheuristics or integer programming. Our project instead chains four different algorithm families (flow, approximation, dynamic programming and assignment) so that each stage uses the technique best suited to it, and it adds persistence and emergency replanning in a runnable prototype.

4. Problem Statement and Objectives

Problem Statement: To automatically assign patients to hospitals, operating rooms, surgery start times and beds under capacity, specialization, shift and priority constraints.

Problem Description: Given a set of patients with priority, surgery type, duration, arrival time and ICU need, and a set of hospitals with ORs, beds and surgeons, produce a conflict-free schedule that treats urgent cases early, keeps ICU beds for ICU patients and uses every OR and bed fairly.

Objectives:

Purpose of the project: manual coordination of beds, ORs and surgeons delays urgent care and wastes capacity, so the project builds an algorithmic scheduler that makes these decisions together. The objectives are:

- To route patients to capable hospitals using min-cost maximum flow with separate ICU and General bed capacities, a preferred-hospital penalty and distance-based tie-breaking.
- To balance operating-room workload using randomized quicksort and the Longest Processing Time (LPT) approximation heuristic.
- To sequence surgeries in each OR using subset dynamic programming that respects existing bookings, qualified-surgeon availability, priority and waiting time.
- To assign beds with the Hungarian algorithm, reserving each bed from surgery start until 24 hours after surgery completion and rolling back the surgery if no bed fits.
- To support emergency intake with replanning that preserves procedures already started, and manual patient discharge that releases the bed.
- To provide KMP-based patient/staff name search and KPI and schedule reports (processing rate, average waiting time, makespan, bed reservations, OR utilization).
- To persist hospitals, schedules, reservations and discharges in an H2 database through a Spring REST application, and to verify the algorithms with a repeatable regression suite.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (a dual-core 2 GHz machine is sufficient).
- Memory: 4 GB RAM minimum (8 GB recommended when running the IDE and the Spring application together).
- Storage: about 1 GB of free disk space for the JDK, project, build output and the local H2 database file.
- Display, keyboard and mouse; no GPU or special hardware is required.

Software Requirements:

- Operating system: Windows 10/11 (PowerShell used for the build commands); Linux or macOS also work.
- Java Development Kit (JDK) 17 or later; Java is the implementation language of all algorithms.
- Spring Boot 3.5.x with Spring Web and Spring Data JPA for the REST application; Apache Maven for dependency management.
- H2 Database (file mode, ./data/hospital_scheduler) with the H2 console at /h2-console for local inspection.
- IDE or editor such as IntelliJ IDEA, Eclipse or VS Code; Git for version control.

- A REST client (browser, Postman or curl) to exercise the API, and a terminal for the console program.

6. IMPLEMENTATION

The system is organised into algorithm, model, scheduling and report packages, with a persistence layer for the REST application. Four core modules are shown below. Pipeline order: HospitalRouter (min-cost max-flow) -> ApproxScheduler (randomized quicksort + LPT) -> ORScheduler/DPScheduler (subset DP) -> BedAssigner (Hungarian). The KMP search module is shown at the end.

Module 1: Hospital routing - Min-cost maximum flow

HospitalRouter builds a network of source, patient nodes, General/ICU lanes per hospital and sink. Source-to-patient capacity is 1, and lane-to-sink capacity equals the available beds of that type. Negative-cost residual edges let the algorithm reroute earlier choices, so the routed count is maximised first and cost minimised second.

```
// algorithms/MinCostMaxFlow.java  - hospital routing (successive shortest paths)
public class MinCostMaxFlow {
    static class Edge { int to, cap, rev; long cost;
        Edge(int to, int cap, long cost, int rev) { this.to = to; this.cap = cap; this.cost = cost;
                                                    this.rev = rev; } }
    private final List<List<Edge>> g = new ArrayList<>();
    public MinCostMaxFlow(int n) { for (int i = 0; i < n; i++) g.add(new ArrayList<>()); }


    public void addEdge(int u, int v, int cap, long cost) {
        g.get(u).add(new Edge(v, cap, cost, g.get(v).size()));
        g.get(v).add(new Edge(u, 0, -cost, g.get(u).size() - 1));   // residual edge, negative cost
    }


    /** Returns {maxFlow, minCost}: flow is maximised first, then cost minimised. */
    public long[] run(int s, int t) {
        int n = g.size(); long flow = 0, cost = 0; final long INF = Long.MAX_VALUE / 4;
        while (true) {
            long[] dist = new long[n]; Arrays.fill(dist, INF); dist[s] = 0;
            int[] pv = new int[n], pe = new int[n];
            boolean changed = true;
            for (int it = 0; it < n && changed; it++) {            // Bellman-Ford on residual graph
                changed = false;
                for (int u = 0; u < n; u++) {
                    if (dist[u] == INF) continue;
                    for (int i = 0; i < g.get(u).size(); i++) {
                        Edge e = g.get(u).get(i);
                        if (e.cap > 0 && dist[u] + e.cost < dist[e.to]) {
                            dist[e.to] = dist[u] + e.cost; pv[e.to] = u; pe[e.to] = i; changed = true;
                        }
                    }
                }
            }
            if (dist[t] == INF) break;                              // no augmenting path left
            int f = Integer.MAX_VALUE;
            for (int v = t; v != s; v = pv[v]) f = Math.min(f, g.get(pv[v]).get(pe[v]).cap);
            for (int v = t; v != s; v = pv[v]) {
                Edge e = g.get(pv[v]).get(pe[v]);
                e.cap -= f; g.get(v).get(e.rev).cap += f;
            }
            flow += f; cost += (long) f * dist[t];
        }
        return new long[]{flow, cost};
    }
}
```

Module 2: OR load balancing - Randomized quicksort and LPT

Patients are sorted by descending duration with a random pivot (expected O(n log n)) and each is given to the compatible OR with the smallest accumulated workload, which is the LPT approximation heuristic.

```
// algorithms/RandomizedQuickSort.java + ApproxScheduler.java  - LPT OR load balancing
public class ApproxScheduler {
    private static final Random RND = new Random();


    /** Randomized quicksort: patients by DESCENDING surgery duration (expected O(n log n)). */
    static void sort(List<Patient> a, int lo, int hi) {
        if (lo >= hi) return;
        Collections.swap(a, lo + RND.nextInt(hi - lo + 1), hi);      // random pivot
        int pivot = a.get(hi).getDuration(), i = lo;
        for (int j = lo; j < hi; j++)
            if (a.get(j).getDuration() > pivot) Collections.swap(a, i++, j);
        Collections.swap(a, i, hi);
        sort(a, lo, i - 1); sort(a, i + 1, hi);
    }


    /** LPT: longest case first, always to the least-loaded compatible OR. */
    public Map<Patient, OperatingRoom> balance(List<Patient> patients, List<OperatingRoom> ors) {
        List<Patient> order = new ArrayList<>(patients);
        sort(order, 0, order.size() - 1);
        Map<OperatingRoom, Integer> load = new HashMap<>();
        Map<Patient, OperatingRoom> result = new LinkedHashMap<>();
        for (Patient p : order) {
            OperatingRoom best = null;
            for (OperatingRoom r : ors) {
                if (!r.supports(p.getSurgeryType())) continue;
                if (best == null || load.getOrDefault(r, 0) < load.getOrDefault(best, 0)) best = r;
            }
            if (best == null) { System.out.println("Warning: no OR supports " + p.getName()); continue; }
            load.merge(best, p.getDuration(), Integer::sum);
            result.put(p, best);
        }
        return result;
    }
}
```

Module 3: OR time slots - Subset dynamic programming

For every OR, a bitmask state records which patients are already sequenced. Each transition finds the earliest feasible start after the previous case, existing bookings, the patient's arrival and surgeon availability, and adds a base reward plus priority bonus minus a waiting penalty. This is exact for up to 14 patients per OR; larger lists use a weighted-interval fallback.

```
// algorithms/DPScheduler.java  - subset DP: order the cases of ONE OR (exact for <= 14 patients)
public class DPScheduler {
    static final int BASE = 1000, WAIT_PENALTY = 1;
    static int bonus(Priority pr) {
        return pr == Priority.EMERGENCY ? 600 : pr == Priority.URGENT ? 300 : 0; }


    /** dp[mask] = best (score, endTime) after scheduling exactly the patients in mask. */
    public List<Slot> schedule(List<Patient> ps, OperatingRoom room, int windowStart, int windowEnd) {
        int n = ps.size(), full = 1 << n;
        double[] score = new double[full];
        int[] end = new int[full], last = new int[full], start = new int[full];
        Arrays.fill(score, Double.NEGATIVE_INFINITY); score[0] = 0; end[0] = windowStart;
        for (int mask = 0; mask < full; mask++) {
            if (score[mask] == Double.NEGATIVE_INFINITY) continue;
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) != 0) continue;                       // each patient used once
                Patient p = ps.get(i);
                int s = room.earliestFree(Math.max(end[mask], p.getArrival()), p.getDuration());
                s = surgeonAvailable(p, s, p.getDuration());                 // shift for qualified surgeon
                if (s < 0 || s + p.getDuration() > windowEnd) continue;      // infeasible transition
                double sc = score[mask] + BASE + bonus(p.getPriority())
                          - WAIT_PENALTY * (s - p.getArrival());
```

```
                int nm = mask | (1 << i);
                if (sc > score[nm] || (sc == score[nm] && s + p.getDuration() < end[nm])) {
                    score[nm] = sc; end[nm] = s + p.getDuration(); last[nm] = i; start[nm] = s;
                }
            }
        }
        int best = 0;                                                       // best reachable subset
        for (int m = 1; m < full; m++) if (score[m] > score[best]) best = m;
        LinkedList<Slot> plan = new LinkedList<>();                         // backtrack the sequence
        for (int m = best; m != 0; m ^= 1 << last[m])
            plan.addFirst(new Slot(ps.get(last[m]), start[m], start[m] + ps.get(last[m]).getDuration()));
        return plan;
    }
}
```

Module 4: Bed assignment - Hungarian algorithm

BedAssigner builds a patient-by-bed cost matrix: priority level x 100 minus waiting minutes, plus 1,000 when a non-ICU patient takes an ICU bed. ICU mismatches and overlapping reservation intervals are forbidden edges. Beds are reserved from surgery start until 24 hours after surgery completion; if no bed fits, the OR slot, surgeon slot and scheduled state are rolled back.

```
// algorithms/HungarianAlgorithm.java  - bed assignment (potentials, O(n^3), rectangular + forbidden edges)
public class HungarianAlgorithm {
    static final double BIG = 1e9;


    /** cost[i][j]: cost of bed j for patient i (POSITIVE_INFINITY = forbidden).
     *  Returns the bed index per patient, or -1 if unmatched. */
    public static int[] solve(double[][] cost) {
        int n = cost.length, m = cost[0].length, k = Math.max(n, m);
        double[][] a = new double[k + 1][k + 1];                // pad to square; dummy cells cost 0
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++)
                a[i][j] = Double.isInfinite(cost[i - 1][j - 1]) ? BIG : cost[i - 1][j - 1];
        double[] u = new double[k + 1], v = new double[k + 1];
        int[] p = new int[k + 1], way = new int[k + 1];
        for (int i = 1; i <= k; i++) {
            p[0] = i; int j0 = 0;
            double[] minv = new double[k + 1]; Arrays.fill(minv, Double.MAX_VALUE);
            boolean[] used = new boolean[k + 1];
            do {
                used[j0] = true; int i0 = p[j0], j1 = 0; double delta = Double.MAX_VALUE;
                for (int j = 1; j <= k; j++) if (!used[j]) {
                    double cur = a[i0][j] - u[i0] - v[j];
                    if (cur < minv[j]) { minv[j] = cur; way[j] = j0; }
                    if (minv[j] < delta) { delta = minv[j]; j1 = j; }
                }
                for (int j = 0; j <= k; j++)
                    if (used[j]) { u[p[j]] += delta; v[j] -= delta; } else minv[j] -= delta;
                j0 = j1;
            } while (p[j0] != 0);
            do { int j1 = way[j0]; p[j0] = p[j1]; j0 = j1; } while (j0 != 0);
        }
        int[] res = new int[n]; Arrays.fill(res, -1);
        for (int j = 1; j <= m; j++)
            if (p[j] >= 1 && p[j] <= n && a[p[j]][j] < BIG) res[p[j] - 1] = j - 1;  // forbidden: unmatched
        return res;
    }
}
```

Supporting module: KMP name search

Menu option 7 searches every patient and staff name with KMP in O(n+m) per name.

```
// algorithms/KMPAlgorithm.java  - case-insensitive name search
public static boolean search(String text, String pattern) {
    text = text.toLowerCase(); pattern = pattern.toLowerCase();
    if (pattern.isEmpty()) return true;
    int[] lps = new int[pattern.length()];                  // longest proper prefix that is also a suffix
    for (int i = 1, len = 0; i < pattern.length(); ) {
        if (pattern.charAt(i) == pattern.charAt(len)) lps[i++] = ++len;
```

```
        else if (len > 0) len = lps[len - 1]; else lps[i++] = 0;
    }
    for (int i = 0, j = 0; i < text.length(); ) {            // text pointer never moves backwards
        if (text.charAt(i) == pattern.charAt(j)) { i++; j++; if (j == pattern.length()) return true; }
        else if (j > 0) j = lps[j - 1]; else i++;
    }
    return false;
}
```

Persistence avedules are kept as superseded history. The console program remains an in-memory workflow.

7. RESULTS & DISCUSSION

The console program was run with the menu sequence 3, 5, 6, 0 (run pipeline, show KPIs, show schedule, exit) on the sample dataset. The results below are the values observed in that run.

7.1 Sample dataset

Hospital	Beds (General + ICU)	Operating rooms (surgery types)
H1 City Central Hospital	6 + 3	OR-A: General, ENT, Orthopedic; OR-B: Cardiac; OR-C: Neurological
H2 Suburban Medical	5 + 1	OR-D: Orthopedic, General; OR-E: Ophthalmic, ENT
H3 Regional Trauma Center	3 + 4	Trauma OR-1: General, Orthopedic, Neurological; OR-2: Cardiac, General

Table 1: Hospitals, beds and operating rooms (12 patients: 4 Emergency, 4 Urgent, 4 Elective; 4 need ICU)

7.2 Pipeline output and schedule

```
Choice: 3
Starting Full Scheduling Pipeline...

Step 1: Network Flow -> Routing Patients across Hospitals...
--> Hospital City Central Hospital received 6 patients.
    Step 2: Approximation Algorithm -> Load Balancing across ORs...
    Step 3: Dynamic Programming -> Optimizing Time Slots within ORs...
    Step 4: Hungarian Algorithm -> Assigning Beds to scheduled patients...
--> Hospital Suburban Medical received 3 patients.
    Step 2: Approximation Algorithm -> Load Balancing across ORs...
    Step 3: Dynamic Programming -> Optimizing Time Slots within ORs...
    Step 4: Hungarian Algorithm -> Assigning Beds to scheduled patients...
--> Hospital Regional Trauma Center received 3 patients.
    Step 2: Approximation Algorithm -> Load Balancing across ORs...
    Step 3: Dynamic Programming -> Optimizing Time Slots within ORs...
    Step 4: Hungarian Algorithm -> Assigning Beds to scheduled patients...

Pipeline execution complete.
```

[Paste screenshot here: Console output of menu option 3 (run pipeline) showing routing counts per hospital]

Min-cost maximum flow routed 6 patients to City Central, 3 to Suburban Medical and 3 to Regional Trauma Center. Routing only decides the hospital; OR and start times are fixed in later stages.

[Paste screenshot here: Console output of menu option 6

(sch

```
Choice: 6

============ PATIENT SCHEDULES ============

[City Central Hospital]
  OR-A (General) Schedule:
    11:20 AM - 12:05 PM : Patient P5 (Evan Wright, Urgent; Surgeon: Dr. Smith)
    12:05 PM - 12:55 PM : Patient P11 (Kevin Ford, Elective; Surgeon: Dr. Smith)
    12:55 PM - 1:55 PM : Patient P1 (Alice Brown, Elective; Surgeon: Dr. Smith)
    1:55 PM - 3:40 PM : Patient P9 (Ian Cook, Urgent; Surgeon: Dr. Smith)
  OR-B (Cardiac) Schedule:
    8:20 AM - 11:20 AM : Patient P2 (Bob Smith, Urgent; Surgeon: Dr. Smith)
  OR-C (Neuro) Schedule:
    9:00 AM - 11:00 AM : Patient P3 (Charlie Davis, Emergency; Surgeon: Dr. Jones)

[Suburban Medical]
  OR-D (Ortho) Schedule:
    10:30 AM - 11:45 AM : Patient P7 (George Hill, Urgent; Surgeon: Dr. Miller)
    11:45 AM - 1:15 PM : Patient P4 (Diana Prince, Elective; Surgeon: Dr. Miller)
  OR-E (Eye/ENT) Schedule:
    10:00 AM - 10:30 AM : Patient P8 (Hannah Lee, Elective; Surgeon: Dr. Miller)

[Regional Trauma Center]
  Trauma OR-1 Schedule:
    1:20 PM - 2:50 PM : Patient P12 (Lucy Kane, Emergency; Surgeon: Dr. Williams)
    2:50 PM - 5:20 PM : Patient P10 (Julia Bell, Emergency; Surgeon: Dr. Williams)
  Trauma OR-2 Schedule:
    9:20 AM - 1:20 PM : Patient P6 (Fiona Ray, Emergency; Surgeon: Dr. Williams)
==========================================
```

edule view) listing OR slots, patients and surgeons]

All 12 patients received an OR slot with a qualified surgeon and no overlaps. Emergency cases such as P3 (9:00 AM) and P6 (9:20 AM) start soon after arrival.

Hospital / OR	**Patients scheduled (in order)**
City Central - OR-A	P5 Evan Wright 11:20 AM-12:05 PM; P11 Kevin Ford 12:05-12:55 PM; P1 Alice Brown 12:55-1:55 PM; P9 Ian Cook 1:55-3:40 PM
City Central - OR-B	P2 Bob Smith 8:20-11:20 AM
City Central - OR-C	P3 Charlie Davis 9:00-11:00 AM
Suburban - OR-D	P7 George Hill 10:30-11:45 AM; P4 Diana Prince 11:45 AM-1:15 PM

Hospital / OR	Patients scheduled (in order)
Suburban - OR-E	P8 Hannah Lee 10:00-10:30 AM
Trauma Center - OR-1	P12 Lucy Kane 1:20-2:50 PM; P10 Julia Bell 2:50-5:20 PM
Trauma Center - OR-2	P6 Fiona Ray 9:20 AM-1:20 PM

Table 2: Observed OR assignments (12-hour clock)

7.3 KPI report

[Paste screenshot here: Console

out

```
Choice: 5

============ SYSTEM KPI REPORT ============
Processing Rate: 12/12 patients scheduled (100%)
Avg Waiting Time: 108 minutes
Makespan (Latest completion): 5:20 PM

--- Hospital Resource Utilization ---
City Central Hospital:
  Bed Reservations: 66.7%
  OR Utilization: [OR-A (General): 36.1%] [OR-B (Cardiac): 25.0%] [OR-C (Neuro): 16.7%]
Suburban Medical:
  Bed Reservations: 50.0%
  OR Utilization: [OR-D (Ortho): 22.9%] [OR-E (Eye/ENT): 4.2%]
Regional Trauma Center:
  Bed Reservations: 42.9%
  OR Utilization: [Trauma OR-1: 33.3%] [Trauma OR-2: 33.3%]
==========================================
```

put of menu option 5 (KPI report)]

Every patient was scheduled with a 108-minute average wait, and the last operation finishes at 5:20 PM. Bed reservation and OR utilization are plan ratios, not a real-time census.

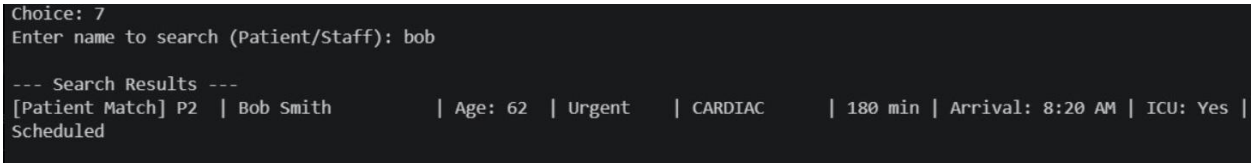
Metric	Observed value
Patients routed (City Central / Suburban / Trauma Center)	6 / 3 / 3
Processing rate	12/12 patients scheduled (100%)
OR slots created	12 (equal to scheduled patients)
Average waiting time	108 minutes
Makespan (latest completion)	5:20 PM
Bed reservations	City Central 66.7%, Suburban 50.0%, Trauma Center 42.9%
OR utilization (08:00-20:00 window)	OR-A 36.1%, OR-B 25.0%, OR-C 16.7%, OR-D 22.9%, OR-E 4.2%, Trauma OR-1 33.3%, Trauma OR-2 33.3%

Table 3: Observed KPI values

7.4 Search, emergency intake and discharge

[Paste screenshot here: Menu option 7 searching "smith"; option 4

emergen



cy intake; option 8 discharge]

KMP search for "smith" matches patient Bob Smith and staff Dr. Smith, case-insensitively. Emergency intake keeps started cases and replans the future ones; discharge releases the bed and keeps the OR slot as history.

7.5 REST application and persistence

Schedule records keep their status, algorithm path and timestamps. After a process restart the 13 patients, 13 schedules and 1 discharge record were still present.

7.6 Verification summary

Verification	Result
Dependency-free algorithm regression suite (flow cost, routing, ICU capacity, Hungarian, KMP, sort/LPT, DP, utilization, bed intervals, validation, emergency)	125 checks passed (recorded before the persistence update)
REST application manual run with H2 persistence	Seeded 3 hospitals and 12 patients; 12 active schedules; data retained after restart
Emergency + discharge + restart	13 patients, 13 schedule records (12 active, 1 completed), 1 discharge record retained
PersistenceIntegrationTests.java	Written but NOT executed: Maven could not resolve the Spring Boot 3.5.16 parent in the restricted environment

Table 4: Testing and verification status

7.7 Discussion

- **Consistency:** the number of scheduled patient flags equals the number of stored OR slots (12/12), so a patient counts as scheduled only when an OR, a qualified surgeon and a bed reservation all exist.

- **Why the stages are separate:** routing to a hospital does not guarantee a feasible start time. A hospital can receive a patient before OR, surgeon and bed constraints determine the actual slot, so each stage is reported separately.
- **Variability:** randomized quicksort can reorder equal-duration cases, so the exact schedule may change between runs while capacity, staff, bed and non-overlap checks remain in force. The schedule should be read from the current run, not treated as fixed.
- **Complexity:** quicksort is expected O(n log n) (worst case O(n^2)); LPT scanning is about O(n*r); subset DP is exponential in the cases per OR and exact up to 14; Hungarian is O(n^3); KMP is O(n+m) per name.
- **Limits of the results:** LPT is a heuristic and no approximation ratio is proved for the full problem; the DP is a per-OR optimum, not a global optimum across hospitals; the bed cost function is hand-designed, not clinically calibrated.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The project set out to schedule hospital routing, operating rooms, surgery times and beds using advanced algorithms, and these objectives were met within the stated simulation scope. Min-cost maximum flow routed all 12 sample patients within separate ICU and General capacities; randomized quicksort with LPT balanced room workload; subset dynamic programming produced conflict-free sequences that respect surgeon shifts, priority and waiting time; and the Hungarian algorithm reserved beds over each full stay interval with rollback when none fits. In the observed run all 12 patients were scheduled (100%), the average wait was 108 minutes and the makespan was 5:20 PM.

Emergency replanning, manual discharge, strict input validation and KMP search were also implemented, and the Spring REST application preserved its data across restarts. The algorithm regression suite passed 125 checks. Two caveats remain: the persistence integration tests were written but not executed, and the system is a classroom simulation, not a clinically validated hospital tool.

Future Scope

Near term

- Run and fix the persistence integration tests once the Spring Boot dependencies can be resolved, and add them to the repeatable test suite.
- Show bed IDs, per-patient unscheduled reasons and a constraint-violation report in the schedule output.
- Let users supply real patient coordinates and configurable post-operative stay lengths instead of the fixed 24-hour assumption.

Medium term

- Model anesthetists, nurses and equipment as extra resources in OR and bed scheduling.
- Make admissions date-aware so that multi-day schedules can be built, and add a web front end with charts.
- Compute a global objective across all hospitals and ORs, and compare LPT and DP results against an optimal baseline with a proved or measured approximation ratio.

Long term

- Integrate real-time hospital data and predict surgery durations and length of stay with machine learning.

- Add adaptive re-optimisation, monitoring, access control, backups and a production-grade database for multi-site deployment, followed by clinical validation.
- Extend the string and algorithm toolkit (for example Aho-Corasick search or parallel scheduling) for larger hospital networks.

9. REFERENCES

[1] B. Cardoen, E. Demeulemeester and J. Beliën, "Operating room planning and scheduling: A literature review," European Journal of Operational Research, vol. 201, no. 3, pp. 921-932, 2010. https://doi.org/10.1016/j.ejor.2009.04.011 (open copy: https://lirias.kuleuven.be/retrieve/17688)

[2] M. Samudra, C. Van Riet, E. Demeulemeester, B. Cardoen, N. Vansteenkiste and F. E. Rademakers, "Scheduling operating rooms: achievements, challenges and pitfalls," Journal of Scheduling, 2016. https://lirias.kuleuven.be/retrieve/380726

[3] E. Demeulemeester, J. Beliën, B. Cardoen and M. Samudra, "Operating Room Planning and Scheduling," in Handbook of Healthcare Operations Management, Springer, 2013, pp. 121-152. https://ideas.repec.org/h/spr/isochp/978-1-4614-5885-2_5.html

[4] P. Demeester, W. Souffriau, P. De Causmaecker and G. Vanden Berghe, "A hybrid tabu search algorithm for automatically assigning patients to beds," Artificial Intelligence in Medicine, vol. 48, no. 1, pp. 61-70, 2010. https://lirias.kuleuven.be/780435

[5] I. Abu Doush, M. A. Al-Betar, M. A. Awadallah and A. I. Hammouri, "Harmony Search Algorithm for Patient Admission Scheduling Problem," Journal of Intelligent Systems, 2018. https://doaj.org/article/760836c9b9174402b2f43f460227b656

[6] "A greedy-tabu approach to the patient bed assignment problem in the Hospital Universitario San Ignacio," Pontificia Universidad Javeriana. https://perfilesycapacidades.javeriana.edu.co/en/publications/a-greedy-tabu-approach-to-the-patient-bed-assignment-problem-in-t/

[7] "Structural Insights and an IP-based Solution Method for Patient-to-room Assignment under Consideration of Single Room Entitlements," arXiv:2401.00221, 2024. https://arxiv.org/pdf/2401.00221

[8] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, pp. 83-97, 1955. https://doi.org/10.1002/nav.3800020109

[9] R. L. Graham, "Bounds on multiprocessor timing anomalies," SIAM Journal on Applied Mathematics, vol. 17, no. 2, pp. 416-429, 1969. https://doi.org/10.1137/0117039

[10] D. E. Knuth, J. H. Morris and V. R. Pratt, "Fast pattern matching in strings," SIAM Journal on Computing, vol. 6, no. 2, pp. 323-350, 1977. https://doi.org/10.1137/0206024

[11] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," Journal of the ACM, vol. 19, no. 2, pp. 248-264, 1972. https://doi.org/10.1145/321694.321699

[12] "Minimum-cost flow problem," Wikipedia. https://en.wikipedia.org/wiki/Minimum-cost_flow_problem

[13] Spring Boot reference documentation. https://spring.io/projects/spring-boot

[14] H2 Database Engine documentation. https://www.h2database.com